

ARRAYS & HASHING

Trade $O(n)$ memory for $O(1)$ lookup — and say the trade out loud.

Contains Duplicate LC 217 $O(n)/O(n)$

Has anything here been seen before?

```
def contains_duplicate_set(nums):
    seen = set()
    for num in nums:
        if num in seen:
            return True
        seen.add(num)
    return False
trap add before check and every element matches itself
```

Two Sum LC 1 $O(n)/O(n)$

The pair hitting a target — return indices, not values.

```
def two_sum(nums, target):
    seen = {}
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []
trap [0,4,0] target 0 — use 'in', never truthiness
```

Valid Anagram LC 242 $O(n)/O(k)$

Same letters, same counts, order irrelevant.

```
def is_anagram_dict(s, t):
    if len(s) != len(t):
        return False
    counts = {}
    for ch in s:
        counts[ch] = counts.get(ch, 0) + 1
    for ch in t:
        counts[ch] = counts.get(ch, 0) - 1
        if counts[ch] < 0:
            return False
    return True
trap "aab" / "abb" — a set-based answer passes it
```

Group Anagrams LC 49 $O(n \cdot k \log k)$

Bucket by a signature you compute, not pairwise.

```
from collections import defaultdict
def group_anagrams(strs):
    groups = defaultdict(list)
    for word in strs:
        key = "".join(sorted(word))
        groups[key].append(word)
    return list(groups.values())
trap sorted() is a list — join it, a list can't be a key
```

Top K Frequent LC 347 $O(n)$ buckets

Count, then rank without sorting.

```
from collections import Counter
def top_k_frequent_buckets(nums, k):
    counts = Counter(nums)
    buckets = [[] for _ in range(len(nums) + 1)]
    for num, freq in counts.items():
        buckets[freq].append(num)
    result = []
    for freq in range(len(buckets) - 1, 0, -1):
        for num in buckets[freq]:
            result.append(num)
            if len(result) == k:
                return result
    return result
trap most_common(k) is right but costs a sort
```

Intersection of Two Arrays II LC 350 $O(n+m)/O(m)$

Multiset — duplicates count, so a set is wrong.

```
from collections import Counter
def intersect(nums1, nums2):
    counts = Counter(nums2)
    result = []
    for value in nums1:
        if counts[value] > 0:
            result.append(value)
            counts[value] -= 1
    return result
trap the II in the title: [1,1] & [1] is [1]
```

ARRAYS & HASHING (CONT.)

Product of Array Except Self LC 238 $O(n)/O(1)^*$

No division — carry a product in from each side.

```
def product_except_self(nums):
    n = len(nums)
    result = [1] * n
    acc = 1
    for i in range(n):
        result[i] = acc
        acc = acc * nums[i]
    acc = 1
    for i in reversed(range(n)):
        result[i] = result[i] * acc
        acc = acc * nums[i]
    return result
trap write the slot before folding nums[i] in
```

Valid Sudoku LC 36 $O(1)$ — 81 cells

Contains Duplicate over 27 groups.

```
EMPTY, SIZE, BOX = "", 9, 3
def is_valid_sudoku_one_pass(board):
    rows = [set() for _ in range(SIZE)]
    cols = [set() for _ in range(SIZE)]
    boxes = [set() for _ in range(SIZE)]
    for r in range(SIZE):
        for c in range(SIZE):
            value = board[r][c]
            if value == EMPTY:
                continue
            box = (r // BOX) * BOX + c // BOX
            if value in rows[r] or value in cols[c] or value
in boxes[box]:
                return False
            rows[r].add(value)
            cols[c].add(value)
            boxes[box].add(value)
    return True
trap a dup inside a box only; skip "", before the sets
```

TWO POINTERS

Two indices instead of a nested loop — needs sorted or symmetric input.

Valid Palindrome LC 125 $O(n)/O(1)$

Inward from both ends, skipping what doesn't count.

```
def is_palindrome_while(s):
    left, right = 0, len(s) - 1
    while left < right:
        while left < right and not s[left].isalnum():
            left = left + 1
        while left < right and not s[right].isalnum():
            right = right - 1
        if s[left].lower() != s[right].lower():
            return False
        left += 1
        right -= 1
    return True
trap the skip must repeat l < r, or "", runs off the end
```

Two Sum II — Sorted LC 167 $O(n)/O(1)$

Sorted input: the sum says which pointer moves.

```
def two_sum_sorted(numbers, target):
    left, right = 0, len(numbers) - 1
    while left < right:
        total = numbers[left] + numbers[right]
        if total == target:
            return [left + 1, right + 1]
        if total < target:
            left += 1
        else:
            right -= 1
    return []
trap unsorted input is silently wrong, not an error
```

SLIDING WINDOW

A contiguous run with a property, in one pass. Watch the shrink.

Longest Substring No Repeat LC 3 $O(n)/O(k)$

Jump the window start, don't crawl it.

```
def longest_substr_no_repeat(s):
    last_seen = {}
    window_start = 0
    longest = 0
    for window_end, ch in enumerate(s):
        if ch in last_seen:
            window_start = max(last_seen[ch] + 1,
window_start)
        last_seen[ch] = window_end
        longest = max(longest, window_end - window_start + 1)
    return longest
trap no max() and "abba" returns 3; +1, window inclusive
```

STACK & QUEUE

Nesting and order. Bracket parsing is XML and JSON validation renamed.

Valid Parentheses LC 20 $O(n)/O(n)$

Most-recently-opened closes first; dict keyed by the closer.

```
def valid_parentheses(s):
    stack = []
    pairs = {"(": ")", "}": "{", "]": "["}
    for ch in s:
        if ch in pairs:
            if not stack or stack[-1] != pairs[ch]:
                return False
            stack.pop()
        else:
            stack.append(ch)
    return not stack
trap 'return True' passes ""(!); empty check first
```

Min Stack LC 155 $O(1)$ per op

$O(1)$ minimum — store it, don't recompute it.

```
class MinStack:
    def __init__(self):
        self.stack = []
    def push(self, val):
        smallest = val if not self.stack else min(val,
self.stack[-1][1])
    self.stack.append((val, smallest))
    def pop(self):
        self.stack.pop()
    def top(self):
        return self.stack[-1][0]
    def get_min(self):
        return self.stack[-1][1]
trap one min variable cannot survive a pop
```

BINARY SEARCH

Halve the space each step — the same reasoning as git bisect.

Binary Search LC 704 $O(\log n)/O(1)$

Invariant: if it exists, it is in low..high inclusive.

```
def binary_search(nums, target):
    low, high = 0, len(nums) - 1
    while low <= high:
        mid = (low + high) // 2
        if nums[mid] == target:
            return mid
        if nums[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
trap either slip is an infinite loop, not a wrong answer
```

Search a 2D Matrix LC 74 $O(\log m \cdot n)$

A sorted matrix is a sorted array in costume.

```
def search_matrix(matrix, target):
    if not matrix or not matrix[0]:
        return False
    rows, cols = len(matrix), len(matrix[0])
    low, high = 0, rows * cols - 1
    while low <= high:
        mid = (low + high) // 2
        value = matrix[mid // cols][mid % cols]
        if value == target:
            return True
        if value < target:
            low = mid + 1
        else:
            high = mid - 1
    return False
trap guard [] and [[]]; no row relation, walk from top-right
```

LINKED LISTS

Reference structures and the fast/slow pointer.

Linked List Cycle LC 141 $O(n)/O(1)$

Two runners; on a loop the fast one laps the slow.

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    if slow is fast:
        return True
    return False
trap the gap shrinks by exactly 1, so it lands on 0
```

TREES: BFS & DFS

A DOM is a tree. Recursion depth against queue width is the trade.

Maximum Depth LC 104 $O(n)/O(h)$

Base case None; else 1 + the deeper child.

```
from collections import deque
def max_depth(root):
    if root is None:
        return 0
    return 1 + max(max_depth(root.left),
max_depth(root.right))
def max_depth_bfs(root):
    if root is None:
        return 0
    queue = deque([root])
    depth = 0
    while queue:
        depth += 1
        for _ in range(len(queue)):
            node = queue.popleft()
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    return depth
trap limit -1000: a deep chain raises before it's wrong
```

Invert Binary Tree LC 226 $O(n)/O(h)$

Swap the children, then recurse — order is free.

```
def invert_tree(root):
    if root is None:
        return None
    root.left, root.right = root.right, root.left
    invert_tree(root.left)
    invert_tree(root.right)
    return root
trap in place, or a new tree? that changes the space
```

INTERVALS & SORTING

The sort is not a preliminary — it is what makes one greedy pass correct.

Merge Intervals LC 56 $O(n \log n)$

Sort by start, then extend or append.

```
def merge_intervals(interval_list):
    sorted_intervals = sorted(interval_list, key=lambda x:
x[0])
    result = []
    for start, end in sorted_intervals:
        if not result or start > result[-1][1]:
            result.append([start, end])
        else:
            result[-1][1] = max(result[-1][1], end)
    return result
trap max(): [1,10] then [2,3] would shrink it
```

STRING PARSING

No algorithm — phases in a fixed order. The spec is the difficulty.

String to Integer (atoi) LC 8 $O(n)/O(1)$

Four phases; each reads what the last one left.

```
INT_MIN = -(2**31)
INT_MAX = 2**31 - 1
def my_atoi(s):
    i = 0
    n = len(s)
    while i < n and s[i] == " ":
        i += 1
    sign = 1
    if i < n and s[i] in "+-":
        sign = -1 if s[i] == "-" else 1
        i += 1
    value = 0
    while i < n and "0" <= s[i] <= "9":
        value = value * 10 + int(s[i])
        i += 1
    result = sign * value
    if result < INT_MIN:
        return INT_MIN
    if result > INT_MAX:
        return INT_MAX
    return result
trap the bounds are asymmetric and both are reachable
```

THE THREE RULES

Ask about constraints and bad input first.

Write the asserts before the code.

Name the slow solution, then beat it.